

Минобрнауки России
Санкт-Петербургский государственный политехнический университет
Факультет технической кибернетики
Кафедра «Информационные и управляющие системы»

КУРСОВАЯ РАБОТА

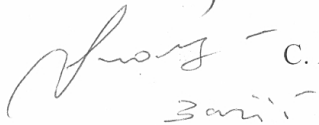
Программирование на ассемблере
по дисциплине: «ЭВМ и периферийные устройства»

Выполнил студент гр. 2084/3

Руководитель

доцент, к.т.н.

 В.К. Костебелова

 С. А. Молодяков

"12" декабря 2011 г.

Санкт-Петербург

2011

Содержание

Введение	3
Программа 1	3-5
Программа 2	5-9
Блок-схема для алгоритма адаптивного планирования	10

Введение

Ассемблер - низкоуровневый машинно-ориентированный язык программирования. Реализация языка зависит от типа процессора и определяется архитектурой вычислительной системы. Ассемблер позволяет напрямую работать с аппаратурой компьютера. Программа на языке ассемблера включает в себя набор команд, которые после трансляции преобразуются в машинные команды.

Цели: 1) Разработать набор процедур для работы с очередью, а именно:

- включение нового элемента;
- выборка очередного элемента (со сдвигом очереди)

2) Организовать мультизадачную среду (3 и более задач) с выводом на экран номера задачи. Переключение происходит по алгоритму адаптивному планированию.

Программы написаны в ASMTTool Professional Edition.

Программа 1

Очередь — структура данных с дисциплиной доступа к элементам «первый пришёл — первый вышел» (FIFO, First In — First Out). Добавление элемента возможно лишь в конец очереди, выборка — только из начала очереди, при этом выбранный элемент из очереди удаляется.

Чтение данных осуществляется через массив. Работа с очередью идет через стек.

```
.model small
.stack 100h
.data

arr dw 10 dup (1,2,3,4,5,6,7,8,9,9)
kolwrite dw 4
kolread dw 3

que_start dw 0 ; начало очереди
que_end dw 0 ; конец очереди
sizes dw 0 ; размер очереди

.code

start:
mov ax,@data
mov ds,ax
xor ax,ax
mov cx,kolwrite
mov bx,offset arr
mov si,cx
add si,kolwrite
add si,si
mov word ptr que_end,si
xor si,si

mov word ptr que_start,sp
; задаем начало и конец очереди
```

```

;запись массива в очередь
;=====
cyc:
    mov ax,word ptr bx[si]
    push ax
    add si,2
loop cyc
    mov cx,kolread
    mov di,kolwrite
    add di,-1
    add di,di
    mov es,sp ; es - begin of queue
;вывод элементов очереди
;=====
cyc2:
    add sp,di
    pop ax
    add di,-2
    mov sp,es
    mov dx,ax
    add dx,30h
    mov ah,2h
    int 21h
loop cyc2

    mov word ptr que_start,sp
    mov di,kolwrite
    sub di,kolread
    mov word ptr sizes,di
    mov si,di
    add si,si
    mov word ptr que_end,di
    ; print size
    ; mov dx,sizes
    ; add dx,30h
    ; mov ah,2h
    ; int 21h
;clear part
;    mov cx, di
;    mov ax,0
;    clearr:
;        pop ax
;        loop clearr
; end of clear part

; DEMO 2
    mov cx,kolwrite
    mov bx,offset arr
    xor si,si
    mov si,kolwrite
    add si,si
;запись остатка массива в очередь
;=====
cycnew:
    mov ax,word ptr bx[si]
    push ax
    add si,2
loop cycnew
    mov si,0
    mov cx,kolread
    ; if not clear part
    mov si,kolwrite

```

```

        sub si,kolread
    ; end of not clear part
        mov di,kolwrite
        add di,si
        add di,-1
        add di,di
        mov es,sp ; es - begin of queue
;Вывод элементов очереди
;=====
cysnew2:
        add sp,di
        pop ax
        add di,-2
        mov sp,es
        mov dx,ax
        add dx,30h
        mov ah,2h
        int 21h
        loop cysnew2
mov     ax,4c00h
int     21h
end     start

```

Программа 2

Мультипланирование предназначено для повышения пропускной способности вычислительной системы путем более равномерной и полной загрузки ее оборудования, в первую очередь процессора.

Основные черты мультипрограммного режима:

- В оперативной памяти находятся несколько пользовательских программ в состоянии активности, ожидания или готовности
- Время работы процессора разделяется между программами, находящимися в памяти в состоянии готовности
- Параллельно с работой процессора происходит подготовка и обмен с несколькими устройствами ввода- вывода

Мультизадачная среда позволяет переключаться от одной запущенной подпрограммы к другой. Для того, чтобы организовать такую среду, можно например заменять различные прерывания. В нашей задаче мы заменяем стандартное прерывание от клавиатуры (int 9h) на свое собственное, которое будет запускать нужную нам подпрограмму.

Алгоритму адаптивного планирования подразумевает, что у каждой подпрограммы есть свой приоритет. Запускается подпрограмма с большим приоритетом, после чего ее приоритет падает на 1. В нашей задаче у первой подпрограммы приоритет 3, у второй – 2, у третьей – 1. Таким образом, смена подпрограмм осуществляется по алгоритму:

121321 – сначала первая, потом вторая, потом снова первая, потом третья и т.д.

Когда приоритет равен 0, то приоритет становится стандартным, который был задан в начале.

Для *сохранения текущего вектора прерывания* воспользуемся int 21h.

Вход: AH = 35h

AL = номер вектора прерывания

Выход: ES:BX = адрес программы обработки прерывания

Описание: Возвращает адрес обработчика прерывания AL. Иными словами, в BX заносится содержимое адреса 0:(4*AL), в ES -0:(4*AL+2).

Для *установки вектора прерывания* воспользуемся прерыванием int 21h.

Вход:

АН = 25H

AL = номер прерывания (от клавиатуры 09h)

DS:DX = вектор прерывания: адрес программы обработки прерывания

Выход: нет

Описание: устанавливает значение элемента таблицы векторов прерываний для прерывания с номером AL равным DS:DX. это равносильно записи 4-байтового адреса в 0000:(AL*4), но, в отличие от прямой записи, DOS здесь знает, что вы делаете, и гарантирует, что в момент записи прерывания будут заблокированы.

.MODEL Tiny

.386

.CODE

thread_struct struc ; структура, где записываем регистры для подпрограмм

_ax dw ? ;0

_bx dw ? ;2

_cx dw ? ;4

_dx dw ? ;6

_ds dw ? ;18

_es dw ? ;20

thread_struct ends

thread1 thread_struct <>

thread2 thread_struct <>

thread3 thread_struct <>

current_thread dw 0 ; номер задачи

ORG 100h

old_handler dd 0 ; старый вектор

rsi dw 1 ; счетчик

rdi dw 2 ; для переключения задач по алгоритму адаптивного планирования

proc1 proc

; задаем начальные регистры из структуры

mov ax, thread1._ax

mov bx, thread1._bx

mov cx, thread1._cx

mov dx, thread1._dx

mov ds, thread1._ds

mov es, thread1._es

mov al, 'I' ; dly vivoda 'I'

; после работы записали регистры снова в нашу структуру

mov thread1._ax, ax

mov thread1._bx, bx

mov thread1._cx, cx

mov thread1._dx, dx

mov thread1._ds, ds

mov thread1._es, es

ret

proc1 endp

proc2 proc

; задаем начальные регистры задачи

mov ax, thread2._ax

mov bx, thread2._bx

mov cx, thread2._cx

```

mov dx, thread2._dx
mov ds, thread2._ds
mov es, thread2._es

mov al,'2'    ; dly vivoda '2'

;после работы записали регистры снова в нашу структуру
mov thread2._ax, ax
mov thread2._bx, bx
mov thread2._cx, cx
mov thread2._dx, dx
mov thread2._ds, ds
mov thread2._es, es
ret
proc2 endp

proc3 proc
; задаем начальные регистры задачи
mov ax,thread3._ax
mov bx,thread3._bx
mov cx, thread3._cx
mov dx, thread3._dx
mov ds, thread3._ds
mov es, thread3._es

mov al,'3'    ; dly vivoda '3'

;после работы записали регистры снова в нашу структуру
mov thread3._ax, ax
mov thread3._bx, bx
mov thread3._cx, cx
mov thread3._dx, dx
mov thread3._ds, ds
mov thread3._es, es
ret
proc3 endp

new_handler proc far

mov si,rsi ;счетчик для переключения
mov di,rdi

add si,1
cmp di,0
je m10
cmp di,2
je m4
cmp si,2
je m8
cmp di,1
je m7
jmp m5
m4:
sub si,2
mov di,2
cmp si,0
je m9
jmp m5
m7:
mov si,1
jmp m5
m8:
mov si,3

```

```

        mov di,2
        jmp m5
m9:
        mov si,1
        mov di,0
        jmp m5
m10:
        mov di,1
        jmp m5
m5:

        mov rsi,si
        mov rdi,di
        mov current_thread,si
        ; print number of task
;
        mov ax,si
;
        mov dx,ax
;
        add dx,30h
;
        mov ah,2h
;
        int 21h

;выход после сх обработок
        add cx,-1
        cmp cx,0
        jle exit ;< or =

        jmp Repeat
Repeat:
        pusha
        cmp si,2
        je m1
        cmp si,1
        je m2
        cmp si,3
        je m6
m1:
        call proc2
        jmp m3
m2:
        call proc1
        jmp m3
m6:
        call proc3
        jmp m3
m3:
        int 29h ; Выводим '1' or '2' or '3'
        popa
        mov al,20h
        out 20h,al

        mov ah,1
        int 16h ; Проверка нажали ли клавишу
        jz Repeat ; Повторяем, если не нажали

        iret
new_handler endp

Start:

        mov ah,35h
        mov al,09h
        int 21h

```



```

mov word ptr old_handler+2,es
mov word ptr old_handler,bx
push cs
pop ds
mov dx,offset new_handler
mov ah,25h
mov al,09h
int 21h

mov thread1._ax, ax
mov thread1._bx, bx
mov thread1._cx, cx
mov thread1._dx, dx
mov thread1._ds, ds
mov thread1._es, es

mov thread2._ax, ax
mov thread2._bx, bx
mov thread2._cx, cx
mov thread2._dx, dx
mov thread2._ds, ds
mov thread2._es, es

mov thread3._ax, ax
mov thread3._bx, bx
mov thread3._cx, cx
mov thread3._dx, dx
mov thread3._ds, ds
mov thread3._es, es

mov current_thread,1

mov cx,8      ; 4 раза сменим обработчик
p:
jmp p

exit:
lds DX,old_handler ;Вернуться к старому обработчику прерывания
mov AH, 25h
mov AL,09h
int 21h
mov ax,4c00h
int 21h
END          Start

```

Блок-схема для алгоритма адаптивного планирования (построенного через метки)

Порядок выбора подпрограмм: 121321 121321 121321

Счетчик si показывает номер подпрограммы, счетчик di показывает переходы.

Например, переключение по возрастанию номера задачи (1-2) di=0.

При обратном переключении (2-1) di=1. При переключении из 1 в 3 si=2, di≠0 и di≠2

При переключении (3-2-1) di=2. При переходе (1-1) у нас становится si=1, di=0 и далее уже будет переход (1-2) после вывода '1'.

Изначально si=1, di=2;

